

## **Ex. No:1 - Basic UNIX Commands**

**Date:**

### **Aim**

To study and practice basic UNIX commands for navigating the terminal, file manipulation, and changing file permissions.

### **A. Navigation Commands**

1. `pwd` - Print the current working directory
2. `ls`, `ls -l`, `ls -a`, `ls -la` - List directory contents (long format, hidden files)
3. `cd <dir>`, `cd ..`, `cd ~`, `cd -` - Change directory (parent, home, previous)
4. `mkdir <dir>` - Create a new directory
5. `rmdir <dir>` - Remove an empty directory

### **B. File Manipulation Commands**

1. `touch file.txt` - Create an empty file / update its timestamp
2. `cat file.txt` - Display file contents
3. `cp source.txt dest.txt` - Copy a file
4. `mv old.txt new.txt` - Rename or move a file
5. `rm file.txt` - Delete a file (`rm -r` to delete a directory recursively)
6. `cat file1 file2 > merged.txt` - Concatenate files into a new file
7. `head -n 5 file.txt` / `tail -n 5 file.txt` - View first/last 5 lines
8. `wc -l file.txt` - Count lines, words, and characters in a file
9. `grep "pattern" file.txt` - Search for a pattern inside a file

### **C. Changing File Permissions**

1. `ls -l file.txt` - View permissions (e.g. `-rwxr--r--`)
2. `chmod 755 file.txt` - Set permissions numerically (owner=`rwx`, group=`rx`, others=`rx`)
3. `chmod u+x file.txt` - Add execute permission for the owner (symbolic form)
4. `chmod g-w file.txt` - Remove write permission for the group
5. `chmod o=r file.txt` - Set 'others' permission to read-only
6. `chown user:group file.txt` - Change file owner and group

### Sample Output

```
$ pwd
/home/student/oslab
$ mkdir test_dir && cd test_dir
$ touch a.txt b.txt
$ ls -l
total 0
-rw-r--r-- 1 student student 0 Jul 16 10:00 a.txt
-rw-r--r-- 1 student student 0 Jul 16 10:00 b.txt
$ chmod 755 a.txt
$ ls -l a.txt

-rwxr-xr-x 1 student student 0 Jul 16 10:00 a.txt
$ echo "Hello OS Lab" > a.txt
$ cat a.txt
Hello OS Lab
$ cp a.txt c.txt
$ mv c.txt d.txt
$ ls
a.txt b.txt d.txt
```

### Result

Basic UNIX navigation, file manipulation, and permission-changing commands were studied and practiced successfully.

## Ex. No. 2 - Implementation of fork(), exec() and wait() System Calls

**Date:**

**Aim**

To write a C program that demonstrates process creation using fork(), replacing a process image using exec(), and process synchronization between parent and child using wait().

**Algorithm**

1. Print the parent process ID using getpid().
2. Create a child process using fork().
3. In the child process, print its PID and parent PID, then replace its process image with the 'ls -l' command using execlp().
4. In the parent process, wait for the child to terminate using wait(&status).
5. Check the exit status of the child using WIFEXITED and WEXITSTATUS macros.
6. Print a completion message from the parent process.

**Program**

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(){
    pid_t pid;
    int status;
    printf("Parent process: PID = %d\n", getpid());
    pid = fork();
    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }
    else if (pid == 0) {
        printf("Child process: PID = %d, Parent PID = %d\n",getpid(), getppid());
        printf("Child is about to execute 'ls -l' using execlp()\n");
        execlp("ls", "ls", "-l", NULL);
        perror("execlp failed");
        exit(1);
    }
    else {
        printf("Parent waiting for child (PID = %d) to finish...\n", pid);
        wait(&status);
        if (WIFEXITED(status))
```

```
        printf("Child terminated normally with exit code %d\n",  
              WEXITSTATUS(status));  
    else  
        printf("Child terminated abnormally\n");  
  
    printf("Parent process finished execution.\n");  
}  
return 0;  
}
```

### Sample Output

```
Parent process: PID = 3369  
Parent waiting for child (PID = 3373) to finish...  
Child process: PID = 3373, Parent PID = 3369  
Child is about to execute 'ls -l' using execlp()  
total 20  
-rwxr-xr-x 1 runner3 runner3 16336 Sep 18 05:37 a.out  
-rw-r--r-- 1 runner3 runner3  968 Sep 18 05:37 main.c  
Child terminated normally with exit code 0  
Parent process finished execution.
```

### Result

The fork(), exec() and wait() system calls were implemented successfully; the child process replaced its image and the parent correctly synchronized with its completion.

### Ex. No. 3 - File Copy using open(), read() and write() System Calls

**Date:**

**Aim**

To write a C program that copies the contents of one file to another using the low-level UNIX system calls open(), read() and write().

**Algorithm**

1. Accept the source and destination file names.
2. Open the source file in read-only mode using open().
3. Create/open the destination file in write mode (O\_CREAT | O\_TRUNC) with permission 0644.
4. Repeatedly read a block of data from the source file using read() and write it to the destination file using write().
5. Continue until read() returns 0 (end of file) or an error.
6. Close both file descriptors and report success.

**Program**

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#define BUF_SIZE 1024
int main(int argc, char *argv[]){
    int src_fd, dest_fd;
    ssize_t n_read;
    char buffer[BUF_SIZE];
    char src_name[100], dest_name[100];
    if (argc == 3) {
        snprintf(src_name, sizeof(src_name), "%s", argv[1]);
        snprintf(dest_name, sizeof(dest_name), "%s", argv[2]);
    } else {
        printf("Enter source file name: ");
        scanf("%99s", src_name);
        printf("Enter destination file name: ");
        scanf("%99s", dest_name);
    }
    src_fd = open(src_name, O_RDONLY);
    if (src_fd < 0) {
        perror("Error opening source file");
        exit(1);
    }
```

```
}
dest_fd = open(dest_name, O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (dest_fd < 0) {
    perror("Error opening destination file");
    close(src_fd);
    exit(1);
}
while ((n_read = read(src_fd, buffer, BUF_SIZE)) > 0) {
    if (write(dest_fd, buffer, n_read) != n_read) {
        perror("Error writing to destination file");
        close(src_fd);
        close(dest_fd);
        exit(1);
    }
}
if (n_read < 0)
    perror("Error reading source file");
else
    printf("File '%s' copied to '%s' successfully.\n", src_name, dest_name);
close(src_fd);
close(dest_fd);
return 0;
}
```

### Sample Output

```
Enter source file name: src.txt
Enter destination file name: dest.txt
File 'src.txt' copied to 'dest.txt' successfully.
```

### Result

The file copy program using open(), read() and write() system calls was executed successfully and the destination file matched the source file exactly.

#### Ex. No. 4 - CPU Scheduling - First Come First Serve (FCFS)

**Date:**

**Aim**

To write a C program to simulate the FCFS CPU scheduling algorithm and compute the waiting time and turnaround time of each process.

**Algorithm**

1. Read the number of processes, and the arrival time & burst time of each process.
2. Process the processes strictly in the order they are given (FCFS order).
3. Maintain a running 'completion time'; if the CPU is idle before the next process arrives, advance the completion time to that arrival time.
4. Waiting time of a process = completion time (before running it) - arrival time.
5. Turnaround time = completion time (after running it) - arrival time.
6. Compute and display the average waiting time and average turnaround time.

**Program**

```
#include <stdio.h>
int main(){
    int n, i;
    int bt[20], at[20], wt[20], tat[20];
    float total_wt = 0, total_tat = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter Arrival Time and Burst Time for each process:\n");
    for (i = 0; i < n; i++) {
        printf("P%d - Arrival Time: ", i + 1);
        scanf("%d", &at[i]);
        printf("P%d - Burst Time: ", i + 1);
        scanf("%d", &bt[i]);
    }
    int completion_time = 0;
    printf("\nProcess\tAT\tBT\tWT\tTAT\n");
    for (i = 0; i < n; i++) {
        if (completion_time < at[i])
            completion_time = at[i];
        wt[i] = completion_time - at[i];
        completion_time += bt[i];
        tat[i] = completion_time - at[i];
        total_wt += wt[i];
    }
```

```
    total_tat += tat[i];  
    printf("P%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], wt[i], tat[i]);  
}  
printf("\nAverage Waiting Time  = %.2f\n", total_wt / n);  
printf("Average Turnaround Time = %.2f\n", total_tat / n);  
  
return 0;  
}
```

### Sample Output

```
Enter number of processes: 3  
Enter Arrival Time and Burst Time for each process:  
P1 - Arrival Time: 0  
P1 - Burst Time: 5  
P2 - Arrival Time: 1  
P2 - Burst Time: 3  
P3 - Arrival Time: 2  
P3 - Burst Time: 8
```

| Process | AT | BT | WT | TAT |
|---------|----|----|----|-----|
| P1      | 0  | 5  | 0  | 5   |
| P2      | 1  | 3  | 4  | 7   |
| P3      | 2  | 8  | 6  | 14  |

```
Average Waiting Time    = 3.33  
Average Turnaround Time = 8.67
```

### Result

The FCFS CPU scheduling algorithm was simulated successfully and the average waiting time and turnaround time were computed correctly.

### Ex. No. 5 - CPU Scheduling - Shortest Job First (SJF), Non-Preemptive

**Date:**

**Aim**

To write a C program to simulate the non-preemptive SJF CPU scheduling algorithm and compute the waiting time and turnaround time of each process.

**Algorithm**

1. Read the number of processes, and the arrival time & burst time of each process.
2. At every scheduling point, consider only the processes that have arrived and are not yet completed.
3. Among these, select the process with the smallest burst time to run next.
4. Run the selected process to completion (non-preemptive) and update its waiting time and turnaround time.
5. If no process has arrived yet, advance the simulated clock (CPU idle).
6. Repeat until all processes are completed, then compute the averages.

**Program**

```
#include <stdio.h>
int main(){
    int n, i;
    int bt[20], at[20], wt[20], tat[20], done[20];
    float total_wt = 0, total_tat = 0;
    int completed = 0, current_time = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter Arrival Time and Burst Time for each process:\n");
    for (i = 0; i < n; i++) {
        printf("P%d - Arrival Time: ", i + 1);
        scanf("%d", &at[i]);
        printf("P%d - Burst Time: ", i + 1);
        scanf("%d", &bt[i]);
        done[i] = 0;
    }
    while (completed != n) {
        int idx = -1;
        int min_bt = 1000000;
        for (i = 0; i < n; i++) {
            if (!done[i] && at[i] <= current_time) {
                if (bt[i] < min_bt) {
```

```
        min_bt = bt[i];
        idx = i;
    }
}

if (idx == -1) {
    current_time++;
    continue;
}
wt[idx] = current_time - at[idx];
current_time += bt[idx];
tat[idx] = current_time - at[idx];
done[idx] = 1;
completed++;
}
printf("\nProcess\tAT\tBT\tWT\tTAT\n");
for (i = 0; i < n; i++) {
    total_wt += wt[i];
    total_tat += tat[i];
    printf("P%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], wt[i], tat[i]);
}
printf("\nAverage Waiting Time   = %.2f\n", total_wt / n);
printf("Average Turnaround Time = %.2f\n", total_tat / n);
return 0;
}
```

### Sample Output

```
Enter number of processes: 4
Enter Arrival Time and Burst Time for each process:
P1 - Arrival Time: 0
P1 - Burst Time: 7
P2 - Arrival Time: 2
P2 - Burst Time: 4
P3 - Arrival Time: 4
P3 - Burst Time: 1
P4 - Arrival Time: 5
P4 - Burst Time: 4
```

| Process | AT | BT | WT | TAT |
|---------|----|----|----|-----|
| P1      | 0  | 7  | 0  | 7   |
| P2      | 2  | 4  | 6  | 10  |
| P3      | 4  | 1  | 3  | 4   |
| P4      | 5  | 4  | 7  | 11  |

Average Waiting Time = 4.00

Average Turnaround Time = 8.00

### Result

The non-preemptive SJF CPU scheduling algorithm was simulated successfully and produces the minimum possible average waiting time for the given process set.

## Ex. No. 6 - CPU Scheduling - Round Robin

**Date:**

**Aim**

To write a C program to simulate the Round Robin CPU scheduling algorithm for a given time quantum and compute the waiting time and turnaround time of each process.

**Algorithm**

1. Read the number of processes, their arrival time, burst time, and the time quantum.
2. Maintain a ready queue. Initially enqueue all processes that arrive at time 0.
3. Repeatedly dequeue the process at the front of the queue and run it for min(time quantum, remaining burst time).
4. If new processes arrive during this time slice, enqueue them before re-queueing the running process (if it has remaining burst time).
5. If the process finishes exactly at or before the quantum expires, record its completion time and compute waiting/turnaround time.
6. Continue until all processes are completed, then compute the averages.

**Program**

```
#include <stdio.h>
int main(){
    int n, i, tq;
    int bt[20], at[20], rem_bt[20], wt[20], tat[20];
    float total_wt = 0, total_tat = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter Arrival Time and Burst Time for each process:\n");
    for (i = 0; i < n; i++) {
        printf("P%d - Arrival Time: ", i + 1);
        scanf("%d", &at[i]);
        printf("P%d - Burst Time: ", i + 1);
        scanf("%d", &bt[i]);
        rem_bt[i] = bt[i];
    }
    printf("Enter Time Quantum: ");
    scanf("%d", &tq);
    int current_time = 0, completed = 0;
    int queue[1000], front = 0, rear = 0;
    int in_queue[20] = {0};
    for (i = 0; i < n; i++) {
        if (at[i] <= current_time && !in_queue[i]) {
```

```
        queue[rear++] = i;
        in_queue[i] = 1;
    }
}

while (completed != n) {
    if (front == rear) {
        current_time++;
        for (i = 0; i < n; i++)
            if (at[i] <= current_time && !in_queue[i] && rem_bt[i] > 0) {
                queue[rear++] = i;
                in_queue[i] = 1;
            }
        continue;
    }
    int idx = queue[front++];
    if (rem_bt[idx] > tq) {
        current_time += tq;
        rem_bt[idx] -= tq;
    } else {
        current_time += rem_bt[idx];
        wt[idx] = current_time - at[idx] - bt[idx];
        tat[idx] = current_time - at[idx];
        rem_bt[idx] = 0;
        completed++;
    }
    for (i = 0; i < n; i++)
        if (at[i] <= current_time && !in_queue[i] && rem_bt[i] > 0) {
            queue[rear++] = i;
            in_queue[i] = 1;
        }
    if (rem_bt[idx] > 0)
        queue[rear++] = idx;
}
printf("\nProcess\tAT\tBT\tWT\tTAT\n");
for (i = 0; i < n; i++) {
    total_wt += wt[i];
    total_tat += tat[i];
    printf("P%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], wt[i], tat[i]);
}
printf("\nAverage Waiting Time   = %.2f\n", total_wt / n);
printf("Average Turnaround Time = %.2f\n", total_tat / n);
return 0;
}
```

## Sample Output

```
Enter number of processes: 4
Enter Arrival Time and Burst Time for each process:
P1 - Arrival Time: 0
P1 - Burst Time: 5
P2 - Arrival Time: 1
P2 - Burst Time: 4
P3 - Arrival Time: 2
P3 - Burst Time: 2
P4 - Arrival Time: 3
P4 - Burst Time: 1
Enter Time Quantum: 2
```

| Process | AT | BT | WT | TAT |
|---------|----|----|----|-----|
| P1      | 0  | 5  | 7  | 12  |
| P2      | 1  | 4  | 6  | 10  |
| P3      | 2  | 2  | 2  | 4   |
| P4      | 3  | 1  | 5  | 6   |

```
Average Waiting Time    = 5.00
Average Turnaround Time = 8.00
```

## Result

The Round Robin CPU scheduling algorithm was simulated successfully for the given time quantum, with fair time-sliced execution among all processes.

### Ex. No. 7 - CPU Scheduling - Priority Scheduling (Non-Preemptive)

**Date:**

**Aim**

To write a C program to simulate the non-preemptive priority CPU scheduling algorithm and compute the waiting time and turnaround time of each process.

#### Algorithm

1. Read the number of processes along with arrival time, burst time and priority (a smaller number denotes a higher priority) of each.
2. At every scheduling point, consider processes that have arrived and are not yet completed.
3. Select the process with the highest priority (smallest priority number); break ties using arrival time.
4. Run the selected process to completion and compute its waiting time and turnaround time.
5. If no process has arrived yet, advance the simulated clock.
6. Repeat until all processes are completed, then compute the averages.

#### Program

```
#include <stdio.h>
int main(){
    int n, i;
    int bt[20], at[20], pr[20], wt[20], tat[20], done[20];
    float total_wt = 0, total_tat = 0;
    int completed = 0, current_time = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter Arrival Time, Burst Time and Priority (lower value = higher priority):\n");
    for (i = 0; i < n; i++) {
        printf("P%d - Arrival Time: ", i + 1);
        scanf("%d", &at[i]);
        printf("P%d - Burst Time: ", i + 1);
        scanf("%d", &bt[i]);
        printf("P%d - Priority: ", i + 1);
        scanf("%d", &pr[i]);
        done[i] = 0;
    }
    while (completed != n) {
        int idx = -1;
        int best_pr = 1000000;
        for (i = 0; i < n; i++) {
            if (!done[i] && at[i] <= current_time) {
```

```
        if (pr[i] < best_pr ||  
            (pr[i] == best_pr && idx != -1 && at[i] < at[idx])) {  
            best_pr = pr[i];  
            idx = i;  
        }  
    }  
}  
  
if (idx == -1) {  
    current_time++;  
    continue;  
}  
wt[idx] = current_time - at[idx];  
current_time += bt[idx];  
tat[idx] = current_time - at[idx];  
done[idx] = 1;  
completed++;  
}  
printf("\nProcess\tAT\tBT\tPriority\tWT\tTAT\n");  
for (i = 0; i < n; i++) {  
    total_wt += wt[i];  
    total_tat += tat[i];  
    printf("P%d\t%d\t%d\t%d\t\t%d\t%d\n", i + 1, at[i], bt[i], pr[i], wt[i], tat[i]);  
}  
printf("\nAverage Waiting Time   = %.2f\n", total_wt / n);  
printf("Average Turnaround Time = %.2f\n", total_tat / n);  
return 0;  
}
```

### Sample Output

```
Enter number of processes: 4
Enter Arrival Time, Burst Time and Priority (lower value = higher priority):
P1 - Arrival Time: 0
P1 - Burst Time: 4
P1 - Priority: 2
P2 - Arrival Time: 1
P2 - Burst Time: 3
P2 - Priority: 3
P3 - Arrival Time: 2
P3 - Burst Time: 5
P3 - Priority: 4
P4 - Arrival Time: 3
P4 - Burst Time: 2
P4 - Priority: 2
```

| Process | AT | BT | Priority | WT | TAT |
|---------|----|----|----------|----|-----|
| P1      | 0  | 4  | 2        | 0  | 4   |
| P2      | 1  | 3  | 3        | 5  | 8   |
| P3      | 2  | 5  | 4        | 7  | 12  |
| P4      | 3  | 2  | 2        | 1  | 3   |

```
Average Waiting Time    = 3.25
Average Turnaround Time = 6.75
```

### Result

The non-preemptive priority CPU scheduling algorithm was simulated successfully, executing higher-priority processes first among the arrived processes.

## Ex. No. 8 - Process Synchronization - Producer-Consumer Problem using Semaphores

**Date:**

### Aim

To write a C program using POSIX threads and semaphores to solve the bounded-buffer Producer-Consumer problem, ensuring mutual exclusion and correct synchronization between the producer and consumer.

### Algorithm

1. Declare a bounded circular buffer of fixed size along with two counting semaphores: 'empty' (initialized to buffer size) and 'full' (initialized to 0), and a mutex lock.
2. Producer: wait on 'empty', lock the mutex, insert an item into the buffer, unlock the mutex, and signal 'full'.
3. Consumer: wait on 'full', lock the mutex, remove an item from the buffer, unlock the mutex, and signal 'empty'.
4. Create the producer and consumer as separate POSIX threads using `pthread_create()`.
5. Join both threads using `pthread_join()` and destroy the semaphores and mutex.

### Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>
#define BUFFER_SIZE 5
#define ITEMS_TO_PRODUCE 10
int buffer[BUFFER_SIZE];
int in = 0, out = 0;
sem_t empty; /* counts empty slots */
sem_t full; /* counts filled slots */
pthread_mutex_t mutex; /* protects buffer access */
void *producer(void *arg){
    int item;
    for (item = 1; item <= ITEMS_TO_PRODUCE; item++) {
        sem_wait(&empty); /* wait for an empty slot */
        pthread_mutex_lock(&mutex); /* enter critical section */
        buffer[in] = item;
        printf("Producer produced item %d at slot %d\n", item, in);
        in = (in + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex); /* leave critical section */
        sem_post(&full); /* signal a new filled slot */
    }
}
```

```
    usleep(100000); /* simulate production time */
}
return NULL;
}

void *consumer(void *arg){
    int item;
    for (int count = 1; count <= ITEMS_TO_PRODUCE; count++) {
        sem_wait(&full);          /* wait for a filled slot */
        pthread_mutex_lock(&mutex); /* enter critical section */
        item = buffer[out];
        printf("\tConsumer consumed item %d from slot %d\n", item, out);
        out = (out + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex); /* leave critical section */
        sem_post(&empty);          /* signal a new empty slot */
        usleep(150000); /* simulate consumption time */
    }
    return NULL;
}

int main(){
    pthread_t prod_thread, cons_thread;
    sem_init(&empty, 0, BUFFER_SIZE); /* initially all slots empty */
    sem_init(&full, 0, 0);           /* initially no slots filled */
    pthread_mutex_init(&mutex, NULL);
    pthread_create(&prod_thread, NULL, producer, NULL);
    pthread_create(&cons_thread, NULL, consumer, NULL);
    pthread_join(prod_thread, NULL);
    pthread_join(cons_thread, NULL);
    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);
    printf("\nAll items produced and consumed successfully.\n");
    return 0;
}
```

## Sample Output

```
Producer produced item 1 at slot 0
    Consumer consumed item 1 from slot 0
Producer produced item 2 at slot 1
    Consumer consumed item 2 from slot 1
Producer produced item 3 at slot 2
Producer produced item 4 at slot 3
    Consumer consumed item 3 from slot 2
Producer produced item 5 at slot 4
    Consumer consumed item 4 from slot 3
Producer produced item 6 at slot 0
Producer produced item 7 at slot 1
    Consumer consumed item 5 from slot 4
Producer produced item 8 at slot 2
    Consumer consumed item 6 from slot 0
Producer produced item 9 at slot 3
Producer produced item 10 at slot 4
    Consumer consumed item 7 from slot 1
    Consumer consumed item 8 from slot 2
    Consumer consumed item 9 from slot 3
    Consumer consumed item 10 from slot 4

All items produced and consumed successfully.
```

## Result

The Producer-Consumer problem was solved successfully using POSIX semaphores and threads, with no buffer overflow, underflow, or race conditions observed.

## Ex. No. 9 - Process Synchronization - Dining Philosophers Problem using Semaphores

**Date:**

**Aim**

To write a C program using POSIX threads and semaphores to solve the Dining Philosophers problem while avoiding deadlock.

**Algorithm**

1. Represent each of the 5 forks as a binary semaphore, initialized to 1 (available).
2. Introduce an additional counting semaphore 'room' initialized to N-1 (one less than the number of philosophers) to prevent all philosophers from picking up a fork simultaneously - this breaks the circular-wait condition and avoids deadlock.
3. Each philosopher thread repeatedly: thinks, then waits on 'room', picks up the left fork semaphore, picks up the right fork semaphore, eats, releases both fork semaphores, and finally releases 'room'.
4. Create all philosopher threads using pthread\_create() and join them using pthread\_join().
5. Destroy all semaphores after all philosophers have finished eating the required number of times.

**Program**

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>
#define N 5
#define MEALS 3
sem_t forks[N];
sem_t room;
void think(int id){
    printf("Philosopher %d is thinking.\n", id);
    usleep(100000);
}
void eat(int id){
    printf("Philosopher %d is eating.\n", id);
    usleep(100000);
}
void *philosopher(void *arg){
    int id = *(int *)arg;
    int left = id;
    int right = (id + 1) % N;
    for (int m = 0; m < MEALS; m++) {
```

```
    think(id);
    sem_wait(&room);
    sem_wait(&forks[left]);
    printf("Philosopher %d picked up left fork %d\n", id, left);
    sem_wait(&forks[right]);
    printf("Philosopher %d picked up right fork %d\n", id, right);

    eat(id);
    sem_post(&forks[right]);
    sem_post(&forks[left]);
    printf("Philosopher %d put down both forks.\n", id);
    sem_post(&room);
}
return NULL;
}
int main(){
    pthread_t phil[N];
    int ids[N];
    for (int i = 0; i < N; i++)
        sem_init(&forks[i], 0, 1);
    sem_init(&room, 0, N - 1);
    for (int i = 0; i < N; i++) {
        ids[i] = i;
        pthread_create(&phil[i], NULL, philosopher, &ids[i]);
    }
    for (int i = 0; i < N; i++)
        pthread_join(phil[i], NULL);
    for (int i = 0; i < N; i++)
        sem_destroy(&forks[i]);
    sem_destroy(&room);
    printf("\nAll philosophers have finished eating. No deadlock occurred.\n");
    return 0;
}
```

## Sample Output

```
Philosopher 0 is thinking.
Philosopher 1 is thinking.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 0 picked up left fork 0
Philosopher 0 picked up right fork 1
Philosopher 0 is eating.
Philosopher 2 picked up left fork 2
Philosopher 3 picked up left fork 3
Philosopher 3 picked up right fork 4
Philosopher 3 is eating.
Philosopher 0 put down both forks.
Philosopher 1 picked up left fork 1
Philosopher 0 is thinking.
Philosopher 3 put down both forks.
Philosopher 3 is thinking.
Philosopher 2 picked up right fork 3
Philosopher 2 is eating.
Philosopher 4 picked up left fork 4
Philosopher 4 picked up right fork 0
Philosopher 4 is eating.
Philosopher 2 put down both forks.
Philosopher 2 is thinking.
Philosopher 1 picked up right fork 2
Philosopher 1 is eating.
Philosopher 4 put down both forks.
Philosopher 4 is thinking.
Philosopher 0 picked up left fork 0
Philosopher 3 picked up left fork 3
Philosopher 3 picked up right fork 4
Philosopher 3 is eating.
Philosopher 1 put down both forks.
.....
Philosopher 1 picked up left fork 1
Philosopher 1 picked up right fork 2
Philosopher 1 is eating.
Philosopher 3 picked up left fork 3
Philosopher 0 is thinking.
Philosopher 1 put down both forks.
Philosopher 1 is thinking.
Philosopher 3 picked up right fork 4
Philosopher 3 is eating.
Philosopher 4 put down both forks.
Philosopher 4 is thinking.
Philosopher 0 picked up left fork 0
Philosopher 0 picked up right fork 1
Philosopher 0 is eating.
Philosopher 2 picked up left fork 2
Philosopher 3 put down both forks.
Philosopher 2 picked up right fork 3
Philosopher 2 is eating.
Philosopher 0 put down both forks.
Philosopher 4 picked up left fork 4
Philosopher 4 picked up right fork 0
Philosopher 4 is eating.
Philosopher 1 picked up left fork 1
Philosopher 2 put down both forks.
Philosopher 1 picked up right fork 2
Philosopher 1 is eating.
Philosopher 4 put down both forks.
Philosopher 1 put down both forks.

All philosophers have finished eating. No deadlock occurred.
```

## Result

The Dining Philosophers problem was solved successfully using semaphores; the 'room' semaphore prevented circular wait, and no deadlock occurred during execution.

## Ex. No. 10 - Deadlock Avoidance - Banker's Algorithm

**Date:**

**Aim**

To write a C program to implement the Banker's Algorithm for deadlock avoidance - to check whether a system is in a safe state and to process resource requests from processes safely.

### Algorithm

1. Read the Allocation matrix, Maximum matrix and Available resources vector for P processes and R resource types.
2. Compute the Need matrix as  $\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$ .
3. Safety algorithm: initialize  $\text{Work} = \text{Available}$  and  $\text{Finish}[i] = \text{false}$  for all processes. Repeatedly find a process i with  $\text{Finish}[i] = \text{false}$  and  $\text{Need}[i] \leq \text{Work}$ ; if found, add its Allocation to Work, mark  $\text{Finish}[i] = \text{true}$ , and add it to the safe sequence.
4. If all processes are marked finished, the system is in a safe state; otherwise it is unsafe (deadlock may occur).
5. Resource-request algorithm: for a request from process P, verify  $\text{Request} \leq \text{Need}$  and  $\text{Request} \leq \text{Available}$ ; tentatively allocate the resources, re-run the safety algorithm, and grant the request only if the resulting state is safe (otherwise roll back and deny it).

### Program

```
#include <stdio.h>
#define P 5
#define R 3
int allocation[P][R];
int max[P][R];
int need[P][R];
int available[R];
void calculate_need(){
    int i, j;
    for (i = 0; i < P; i++)
        for (j = 0; j < R; j++)
            need[i][j] = max[i][j] - allocation[i][j];
}
int is_safe(int safe_seq[]){
    int work[R];
    int finish[P] = {0};
    int i, j, count = 0;
    for (i = 0; i < P; i++)
        work[i] = available[i];
    while (count < P) {
```

```
int found = 0;
for (i = 0; i < P; i++) {
    if (!finish[i]) {
        int can_allocate = 1;
        for (j = 0; j < R; j++) {
            if (need[i][j] > work[j]) {
                can_allocate = 0;
                break;
            }
        }
        if (can_allocate) {
            for (j = 0; j < R; j++)
                work[j] += allocation[i][j];
            safe_seq[count++] = i;
            finish[i] = 1;
            found = 1;
        }
    }
}
if (!found)
    return 0; /* no process could proceed -> unsafe */
return 1;
}

int request_resources(int p_id, int request[R]){
    int i;
    int safe_seq[P];
    for (i = 0; i < R; i++) {
        if (request[i] > need[p_id][i]) {
            printf("Error: Process P%d has exceeded its maximum claim.\n", p_id);
            return 0;
        }
    }
    for (i = 0; i < R; i++) {
        if (request[i] > available[i]) {
            printf("Process P%d must wait; resources are not available.\n", p_id);
            return 0;
        }
    }
    for (i = 0; i < R; i++) {
        available[i] -= request[i];
        allocation[p_id][i] += request[i];
        need[p_id][i] -= request[i];
    }
}
```

```
if (is_safe(safe_seq)) {
    printf("Request can be granted safely.\nNew safe sequence: ");
    for (i = 0; i < P; i++)
        printf("P%d ", safe_seq[i]);
    printf("\n");
    return 1;
} else {
    for (i = 0; i < R; i++) {
        available[i] += request[i];
        allocation[p_id][i] -= request[i];
        need[p_id][i] += request[i];
    }
    printf("Request denied: granting it would leave the system in an unsafe state.\n");
    return 0;
}
}
int main() {
    int i, j;
    printf("Enter Allocation matrix (%d processes x %d resources):\n", P, R);
    for (i = 0; i < P; i++) {
        printf("P%d: ", i);
        for (j = 0; j < R; j++)
            scanf("%d", &allocation[i][j]);
    }
    printf("Enter Maximum matrix (%d processes x %d resources):\n", P, R);
    for (i = 0; i < P; i++) {
        printf("P%d: ", i);
        for (j = 0; j < R; j++)
            scanf("%d", &max[i][j]);
    }
    printf("Enter Available resources vector (%d resources):\n", R);
    for (j = 0; j < R; j++)
        scanf("%d", &available[j]);
    calculate_need();
    printf("\nNeed matrix:\n");
    for (i = 0; i < P; i++) {
        printf("P%d: ", i);
        for (j = 0; j < R; j++)
            printf("%d ", need[i][j]);
        printf("\n");
    }
    int safe_seq[P];
    if (is_safe(safe_seq)) {
        printf("\nSystem is in a SAFE state.\nSafe sequence: ");
    }
}
```

```
    for (i = 0; i < P; i++)
        printf("P%d ", safe_seq[i]);
    printf("\n");
} else {
    printf("\nSystem is in an UNSAFE state (deadlock may occur).\n");
    return 0;
}
int p_id;
int request[R];
printf("\nEnter process number requesting resources: ");
scanf("%d", &p_id);
printf("Enter request vector (%d resources): ", R);
for (j = 0; j < R; j++)
    scanf("%d", &request[j]);
request_resources(p_id, request);
return 0;
}
```

### Sample Output

```
Enter Allocation matrix (5 processes x 3 resources):
P0: 0 1 0
P1: 2 0 0
P2: 3 0 2
P3: 2 1 1
P4: 0 0 2
Enter Maximum matrix (5 processes x 3 resources):
P0: 7 5 3
P1: 3 2 2
P2: 9 0 2
P3: 2 2 2
P4: 4 3 3
Enter Available resources vector (3 resources):
3 3 2

Need matrix:
P0: 7 4 3
P1: 1 2 2
P2: 6 0 0
P3: 0 1 1
P4: 4 3 1

System is in a SAFE state.
Safe sequence: P1 P3 P4 P0 P2

Enter process number requesting resources: 1
Enter request vector (3 resources): 1 0 2
Request can be granted safely.
New safe sequence: P1 P3 P4 P0 P2
```

### Result

The Banker's Algorithm correctly identified the system as being in a safe state with sequence <P1, P3, P4, P0, P2>, and correctly evaluated a resource request without leading to deadlock. These results match the standard textbook example.

### Ex. No. 11 - Memory Management - Contiguous Allocation (First Fit, Best Fit, Worst Fit)

**Date:**

**Aim**

To write a C program to simulate First Fit, Best Fit and Worst Fit contiguous memory allocation strategies for a given set of memory blocks and processes.

**Algorithm**

1. Read the number and sizes of the available memory blocks (partitions), and the number and sizes of the processes to be allocated.
2. First Fit: for each process, scan the blocks in order and allocate it to the first block whose size is greater than or equal to the process size.
3. Best Fit: for each process, scan all blocks and allocate it to the smallest block that is still large enough (minimizes leftover space).
4. Worst Fit: for each process, scan all blocks and allocate it to the largest available block (maximizes leftover space).
5. For each strategy, reduce the chosen block's remaining size by the allocated process size and display the block-to-process allocation table.

**Program**

```
#include <stdio.h>
#include <string.h>
#define MAX 20
void first_fit(int bsize[], int nb, int psize[], int np){
    int block[MAX], allocation[MAX];
    int i, j;
    for (i = 0; i < nb; i++) block[i] = bsize[i]; /* remaining size of each block */
    for (i = 0; i < np; i++) allocation[i] = -1;
    for (i = 0; i < np; i++) {
        for (j = 0; j < nb; j++) {
            if (block[j] >= psize[i]) {
                allocation[i] = j;
                block[j] -= psize[i];
                break;
            }
        }
    }
    printf("\n--- First Fit ---\n");
    printf("Process\tSize\tBlock Allocated\n");
    for (i = 0; i < np; i++) {
        if (allocation[i] != -1)
```

```
        printf("P%d\t%d\tBlock %d\n", i + 1, psize[i], allocation[i] + 1);
    else
        printf("P%d\t%d\tNot Allocated\n", i + 1, psize[i]);
    }
}
```

```
void best_fit(int bsize[], int nb, int psize[], int np){
    int block[MAX], allocation[MAX];
    int i, j;
    for (i = 0; i < nb; i++) block[i] = bsize[i];
    for (i = 0; i < np; i++) allocation[i] = -1;
    for (i = 0; i < np; i++) {
        int best_idx = -1;
        for (j = 0; j < nb; j++) {
            if (block[j] >= psize[i]) {
                if (best_idx == -1 || block[j] < block[best_idx])
                    best_idx = j;
            }
        }
        if (best_idx != -1) {
            allocation[i] = best_idx;
            block[best_idx] -= psize[i];
        }
    }
}
```

```
printf("\n--- Best Fit ---\n");
printf("Process\tSize\tBlock Allocated\n");
for (i = 0; i < np; i++) {
    if (allocation[i] != -1)
        printf("P%d\t%d\tBlock %d\n", i + 1, psize[i], allocation[i] + 1);
    else
        printf("P%d\t%d\tNot Allocated\n", i + 1, psize[i]);
}
}
```

```
void worst_fit(int bsize[], int nb, int psize[], int np){
    int block[MAX], allocation[MAX];
    int i, j;

    for (i = 0; i < nb; i++) block[i] = bsize[i];
    for (i = 0; i < np; i++) allocation[i] = -1;

    for (i = 0; i < np; i++) {
        int worst_idx = -1;
```

```
    for (j = 0; j < nb; j++) {
        if (block[j] >= psize[i]) {
            if (worst_idx == -1 || block[j] > block[worst_idx])
                worst_idx = j;
        }
    }
    if (worst_idx != -1) {
        allocation[i] = worst_idx;
        block[worst_idx] -= psize[i];
    }
}

printf("\n--- Worst Fit ---\n");
printf("Process\tSize\tBlock Allocated\n");
for (i = 0; i < np; i++) {
    if (allocation[i] != -1)
        printf("P%d\t%d\tBlock %d\n", i + 1, psize[i], allocation[i] + 1);
    else
        printf("P%d\t%d\tNot Allocated\n", i + 1, psize[i]);
}
}

int main()
{
    int nb, np, i;
    int bsize[MAX], psize[MAX];
    printf("Enter number of memory blocks: ");
    scanf("%d", &nb);
    printf("Enter size of each block:\n");
    for (i = 0; i < nb; i++) {
        printf("Block %d: ", i + 1);
        scanf("%d", &bsize[i]);
    }
    printf("Enter number of processes: ");
    scanf("%d", &np);
    printf("Enter size of each process:\n");
    for (i = 0; i < np; i++) {
        printf("Process %d: ", i + 1);
        scanf("%d", &psize[i]);
    }
    first_fit(bsize, nb, psize, np);
    best_fit(bsize, nb, psize, np);
    worst_fit(bsize, nb, psize, np);
    return 0;
}
```

## Sample Output

```
Enter number of memory blocks: 5
Enter size of each block:
Block 1: 100
Block 2: 500
Block 3: 200
Block 4: 300
Block 5: 600
Enter number of processes: 4
Enter size of each process:
Process 1: 212
Process 2: 417
Process 3: 112
Process 4: 426
```

### --- First Fit ---

| Process | Size | Block Allocated |
|---------|------|-----------------|
| P1      | 212  | Block 2         |
| P2      | 417  | Block 5         |
| P3      | 112  | Block 2         |
| P4      | 426  | Not Allocated   |

### --- Best Fit ---

| Process | Size | Block Allocated |
|---------|------|-----------------|
| P1      | 212  | Block 4         |
| P2      | 417  | Block 2         |
| P3      | 112  | Block 3         |
| P4      | 426  | Block 5         |

### --- Worst Fit ---

| Process | Size | Block Allocated |
|---------|------|-----------------|
| P1      | 212  | Block 5         |
| P2      | 417  | Block 2         |
| P3      | 112  | Block 5         |
| P4      | 426  | Not Allocated   |

## Result

First Fit, Best Fit and Worst Fit contiguous memory allocation strategies were simulated successfully; Best Fit allocated all processes, while First Fit and Worst Fit left one process unallocated for the given block/process sizes.

## **Ex. No. 12 - Memory Management - Page Replacement Algorithms (FIFO, LRU, Optimal)**

### **Date:**

### **Aim**

To write a C program to simulate FIFO, LRU and Optimal page replacement algorithms for a given page reference string and number of page frames, and to compute the number of page faults for each.

### **Algorithm**

1. Read the reference string (sequence of page numbers requested) and the number of available page frames.
2. FIFO: maintain the frames as a circular queue; on a page fault, replace the page that has been in memory the longest (oldest, first-in).
3. LRU: on a page fault, replace the page in memory that was used least recently (using a 'last used' timestamp per frame).
4. Optimal: on a page fault, replace the page whose next use is farthest in the future (or that will never be used again).
5. For each algorithm, track hits/faults, print the frame contents after each reference, and report the total number of page faults.

### **Program**

```
#include <stdio.h>
#define MAX 50
int is_present(int frames[], int nf, int page){
    int i;
    for (i = 0; i < nf; i++)
        if (frames[i] == page)
            return 1;
    return 0;
}

void fifo(int ref[], int n, int nf)
{
    int frames[MAX];
    int i, j, faults = 0, next = 0;

    for (i = 0; i < nf; i++) frames[i] = -1;

    printf("\n--- FIFO Page Replacement ---\n");
```

```
for (i = 0; i < n; i++) {
    if (!is_present(frames, nf, ref[i])) {
        frames[next] = ref[i];
        next = (next + 1) % nf;
        faults++;
        printf("Page %d -> Fault \tFrames: ", ref[i]);
    } else {
        printf("Page %d -> Hit \tFrames: ", ref[i]);
    }
    for (j = 0; j < nf; j++)
        printf("%d ", frames[j]);
    printf("\n");
}
printf("Total Page Faults (FIFO) = %d\n", faults);
}

void lru(int ref[], int n, int nf)
{
    int frames[MAX], last_used[MAX];
    int i, j, faults = 0;

    for (i = 0; i < nf; i++) { frames[i] = -1; last_used[i] = -1; }

    printf("\n--- LRU Page Replacement ---\n");
    for (i = 0; i < n; i++) {
        if (!is_present(frames, nf, ref[i])) {
            int idx = -1;
            for (j = 0; j < nf; j++)
                if (frames[j] == -1) { idx = j; break; }

            if (idx == -1) {
                int lru_idx = 0;
                for (j = 1; j < nf; j++)
                    if (last_used[j] < last_used[lru_idx])
                        lru_idx = j;
                idx = lru_idx;
            }
            frames[idx] = ref[i];
            last_used[idx] = i;
            faults++;
            printf("Page %d -> Fault \tFrames: ", ref[i]);
        } else {

            for (j = 0; j < nf; j++)
```

```
        if (frames[j] == ref[i]) last_used[j] = i;
        printf("Page %d -> Hit   \tFrames: ", ref[i]);
    }
    for (j = 0; j < nf; j++)
        printf("%d ", frames[j]);
    printf("\n");
}
printf("Total Page Faults (LRU) = %d\n", faults);
}

void optimal(int ref[], int n, int nf)
{
    int frames[MAX];
    int i, j, k, faults = 0;

    for (i = 0; i < nf; i++) frames[i] = -1;

    printf("\n--- Optimal Page Replacement ---\n");
    for (i = 0; i < n; i++) {
        if (!is_present(frames, nf, ref[i])) {
            int idx = -1;
            for (j = 0; j < nf; j++)
                if (frames[j] == -1) { idx = j; break; }

            if (idx == -1) {
                int farthest = -1, replace_idx = 0;
                for (j = 0; j < nf; j++) {
                    int k_use = -1;
                    for (k = i + 1; k < n; k++) {
                        if (frames[j] == ref[k]) { k_use = k; break; }
                    }
                    if (k_use == -1) { replace_idx = j; break; } /* never used again */
                    if (k_use > farthest) { farthest = k_use; replace_idx = j; }
                }
                idx = replace_idx;
            }
            frames[idx] = ref[i];
            faults++;
            printf("Page %d -> Fault \tFrames: ", ref[i]);
        } else {
            printf("Page %d -> Hit   \tFrames: ", ref[i]);
        }
        for (j = 0; j < nf; j++)
            printf("%d ", frames[j]);
    }
}
```

```
        printf("\n");
    }
    printf("Total Page Faults (Optimal) = %d\n", faults);
}

int main()
{
    int ref[MAX], n, nf;

    printf("Enter number of pages in reference string: ");
    scanf("%d", &n);
    printf("Enter the reference string:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &ref[i]);

    printf("Enter number of page frames: ");
    scanf("%d", &nf);

    fifo(ref, n, nf);
    lru(ref, n, nf);
    optimal(ref, n, nf);

    return 0;
}
```

## Sample Output

```
Enter number of pages in reference string: 20
Enter the reference string:
7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
Enter number of page frames: 3
```

```
--- FIFO Page Replacement ---
Page 7 -> Fault      Frames: 7 -1 -1
Page 0 -> Fault      Frames: 7 0 -1
Page 1 -> Fault      Frames: 7 0 1
Page 2 -> Fault      Frames: 2 0 1
Page 0 -> Hit        Frames: 2 0 1
Page 3 -> Fault      Frames: 2 3 1
Page 0 -> Fault      Frames: 2 3 0
Page 4 -> Fault      Frames: 4 3 0
Page 2 -> Fault      Frames: 4 2 0
Page 3 -> Fault      Frames: 4 2 3
Page 0 -> Fault      Frames: 0 2 3
Page 3 -> Hit        Frames: 0 2 3
Page 2 -> Hit        Frames: 0 2 3
Page 1 -> Fault      Frames: 0 1 3
Page 2 -> Fault      Frames: 0 1 2
Page 0 -> Hit        Frames: 0 1 2
Page 1 -> Hit        Frames: 0 1 2
Page 7 -> Fault      Frames: 7 1 2
Page 0 -> Fault      Frames: 7 0 2
Page 1 -> Fault      Frames: 7 0 1
Total Page Faults (FIFO) = 15
```

```
--- LRU Page Replacement ---
Page 7 -> Fault      Frames: 7 -1 -1
Page 0 -> Fault      Frames: 7 0 -1
Page 1 -> Fault      Frames: 7 0 1
Page 2 -> Fault      Frames: 2 0 1
Page 0 -> Hit        Frames: 2 0 1
Page 3 -> Fault      Frames: 2 0 3
Page 0 -> Hit        Frames: 2 0 3
Page 4 -> Fault      Frames: 4 0 3
Page 2 -> Fault      Frames: 4 0 2
Page 3 -> Fault      Frames: 4 3 2
Page 0 -> Fault      Frames: 0 3 2
Page 3 -> Hit        Frames: 0 3 2
Page 2 -> Hit        Frames: 0 3 2
Page 1 -> Fault      Frames: 1 3 2
Page 2 -> Hit        Frames: 1 3 2
Page 0 -> Fault      Frames: 1 0 2
Page 1 -> Hit        Frames: 1 0 2
Page 7 -> Fault      Frames: 1 0 7
Page 0 -> Hit        Frames: 1 0 7
Page 1 -> Hit        Frames: 1 0 7
Total Page Faults (LRU) = 12
```

```
--- Optimal Page Replacement ---  
Page 7 -> Fault      Frames: 7 -1 -1  
Page 0 -> Fault      Frames: 7 0 -1  
Page 1 -> Fault      Frames: 7 0 1  
Page 2 -> Fault      Frames: 2 0 1  
Page 0 -> Hit        Frames: 2 0 1  
Page 3 -> Fault      Frames: 2 0 3  
Page 0 -> Hit        Frames: 2 0 3  
Page 4 -> Fault      Frames: 2 4 3  
Page 2 -> Hit        Frames: 2 4 3  
Page 3 -> Hit        Frames: 2 4 3  
Page 0 -> Fault      Frames: 2 0 3  
Page 3 -> Hit        Frames: 2 0 3  
Page 2 -> Hit        Frames: 2 0 3  
Page 1 -> Fault      Frames: 2 0 1  
Page 2 -> Hit        Frames: 2 0 1  
Page 0 -> Hit        Frames: 2 0 1  
Page 1 -> Hit        Frames: 2 0 1  
Page 7 -> Fault      Frames: 7 0 1  
Page 0 -> Hit        Frames: 7 0 1  
Page 1 -> Hit        Frames: 7 0 1  
Total Page Faults (Optimal) = 9
```

## Result

FIFO, LRU and Optimal page replacement algorithms were simulated successfully for the classic reference string with 3 frames, producing 15, 12 and 9 page faults respectively - confirming that Optimal gives the fewest faults, as expected by Belady's theorem.

### Ex. No. 13 - Disk Scheduling Algorithms (SSTF, SCAN, C-SCAN)

**Date:**

**Aim**

To write a C program to simulate SSTF, SCAN and C-SCAN disk scheduling algorithms for a given set of disk requests and compute the total head movement (seek time) for each.

**Algorithm**

1. Read the disk request queue (cylinder numbers), the initial head position, the disk size, and the initial direction of head movement.
2. SSTF (Shortest Seek Time First): from the current head position, always service the pending request that is closest (minimum seek distance) next.
3. SCAN (Elevator): sort the requests; move the head in the chosen direction servicing all requests in that direction (going all the way to the disk boundary), then reverse direction and service the remaining requests.
4. C-SCAN (Circular SCAN): move the head in one direction servicing all requests up to the disk boundary, then jump directly back to cylinder 0 (this jump is not counted as seek time) and continue servicing the remaining requests in the same direction.
5. For each algorithm, print the seek sequence and compute the total head movement.

**Program**

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
void sstf(int req[], int n, int head)
{
    int visited[MAX] = {0};
    int total = 0, cur = head, i, count = 0;

    printf("\n--- SSTF (Shortest Seek Time First) ---\n");
    printf("Seek Sequence: %d", cur);

    while (count < n) {
        int min_dist = 1000000, idx = -1;
        for (i = 0; i < n; i++) {
            if (!visited[i]) {
                int dist = abs(req[i] - cur);
                if (dist < min_dist) {
                    min_dist = dist;
                    idx = i;
                }
            }
        }
    }
```

```
    }
}
visited[idx] = 1;
total += min_dist;
cur = req[idx];
printf(" -> %d", cur);
count++;
}
printf("\nTotal Head Movement (SSTF) = %d\n", total);
}

int cmp(const void *a, const void *b)
{
    return (*(int *)a - *(int *)b);
}

void scan(int req[], int n, int head, int disk_size, int direction)
{
    int sorted[MAX];
    int i;
    for (i = 0; i < n; i++) sorted[i] = req[i];
    qsort(sorted, n, sizeof(int), cmp);

    int total = 0, cur = head;
    printf("\n--- SCAN (Elevator Algorithm) ---\n");
    printf("Seek Sequence: %d", cur);

    if (direction == 1) { /* move up towards disk_size-1, then reverse */
        for (i = 0; i < n; i++) {
            if (sorted[i] >= head) {
                printf(" -> %d", sorted[i]);
                total += abs(sorted[i] - cur);
                cur = sorted[i];
            }
        }
    }
    if (cur != disk_size - 1) {
        printf(" -> %d", disk_size - 1);
        total += abs((disk_size - 1) - cur);
        cur = disk_size - 1;
    }
    for (i = n - 1; i >= 0; i--) {
        if (sorted[i] < head) {
            printf(" -> %d", sorted[i]);
            total += abs(cur - sorted[i]);
        }
    }
}
```

```
        cur = sorted[i];
    }
}
} else {
    for (i = n - 1; i >= 0; i--) {
        if (sorted[i] <= head) {
            printf(" -> %d", sorted[i]);
            total += abs(cur - sorted[i]);
            cur = sorted[i];
        }
    }
    if (cur != 0) {
        printf(" -> %d", 0);
        total += abs(cur - 0);
        cur = 0;
    }
    for (i = 0; i < n; i++) {
        if (sorted[i] > head) {
            printf(" -> %d", sorted[i]);
            total += abs(sorted[i] - cur);
            cur = sorted[i];
        }
    }
}
printf("\nTotal Head Movement (SCAN) = %d\n", total);
}
```

```
void cscan(int req[], int n, int head, int disk_size)
```

```
{
    int sorted[MAX];
    int i;
    for (i = 0; i < n; i++) sorted[i] = req[i];
    qsort(sorted, n, sizeof(int), cmp);

    int total = 0, cur = head;
    printf("\n--- C-SCAN (Circular SCAN) ---\n");
    printf("Seek Sequence: %d", cur);

    for (i = 0; i < n; i++) {
        if (sorted[i] >= head) {
            printf(" -> %d", sorted[i]);
            total += abs(sorted[i] - cur);
            cur = sorted[i];
        }
    }
}
```

```
}
if (cur != disk_size - 1) {
    printf(" -> %d", disk_size - 1);
    total += abs((disk_size - 1) - cur);
    cur = disk_size - 1;
}
printf(" -> 0 (jump)");
cur = 0;
for (i = 0; i < n; i++) {
    if (sorted[i] < head) {
        printf(" -> %d", sorted[i]);
        total += abs(sorted[i] - cur);
        cur = sorted[i];
    }
}
printf("\nTotal Head Movement (C-SCAN) = %d\n", total);
}

int main()
{
    int n, head, disk_size, i, direction;
    int req[MAX];

    printf("Enter number of disk requests: ");
    scanf("%d", &n);
    printf("Enter the request queue (cylinder numbers):\n");
    for (i = 0; i < n; i++)
        scanf("%d", &req[i]);

    printf("Enter initial head position: ");
    scanf("%d", &head);
    printf("Enter disk size (max cylinder number + 1, e.g. 200): ");
    scanf("%d", &disk_size);
    printf("Enter initial direction for SCAN (1 = towards higher, 0 = towards lower): ");
    scanf("%d", &direction);

    sstf(req, n, head);
    scan(req, n, head, disk_size, direction);
    cscan(req, n, head, disk_size);

    return 0;
}
```

## Sample Output

```
Enter number of disk requests: 8
Enter the request queue (cylinder numbers):
98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter disk size (max cylinder number + 1, e.g. 200): 200
Enter initial direction for SCAN (1 = towards higher, 0 = towards lower): 1

--- SSTF (Shortest Seek Time First) ---
Seek Sequence: 53 -> 65 -> 67 -> 37 -> 14 -> 98 -> 122 -> 124 -> 183
Total Head Movement (SSTF) = 236

--- SCAN (Elevator Algorithm) ---
Seek Sequence: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 37 -> 14
Total Head Movement (SCAN) = 331

--- C-SCAN (Circular SCAN) ---
Seek Sequence: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 0 (jump) -> 14 -> 37
Total Head Movement (C-SCAN) = 183
```

## Result

SSTF, SCAN and C-SCAN disk scheduling algorithms were simulated successfully for the standard example (head=53, 200 cylinders), yielding total head movements of 236, 331 and 183 respectively, matching standard textbook values.